SLIIT
Discover Your Future

# BSc (Hons) in Information Technology
# Year 3

**Lab Sheet 10 – CUDA (Google Colab Edition)**

**SE3082 – Parallel Computing** **Semester 2, 2026**

## Introduction

This lab continues Lab 9. You will run everything on a real NVIDIA GPU in **Google Colab**, using exactly the same *write → compile → run* workflow as before. You will learn how to **measure** how long a GPU program takes, how to make CUDA programs **report their own errors**, and how to cover pictures and volumes with two- and three-dimensional grids.

By the end of this lab you will be able to:

- time a kernel with a CPU timer, and explain why `cudaDeviceSynchronize()` must come before the clock is stopped;
- wrap CUDA calls in a `CHECK()` macro and read the error messages it prints;
- cover an 800 × 600 picture with a 2D grid of 16 × 16 blocks, and a 500 × 500 × 500 volume with a 3D grid;
- write correct bounds checks for one, two and three dimensions.

| Exercise | Topic |
|---|---|
| 1 | Timing a GPU program |
| 2 | Catching errors with the CHECK() macro |
| 3 | Scaling a picture with a 2D grid (PictureKernel) |
| 4 | Multiplying two 3D matrices element by element (3D grid) |

**You need:** a Google account, a web browser, and your working programs from Lab 9. Exercise 1 asks you to time some of them.

### Before you begin – Colab set-up (short recap of Lab 9)

1. Open `colab.research.google.com` and choose **File → New notebook** (or reopen your Lab 9 notebook and add cells to it). Name it **SE3082_Lab10_<your student ID>**.
2. Choose **Runtime → Change runtime type**, select **T4 GPU** and click **Save**.
3. Run this cell. It must show a GPU (normally *Tesla T4*) and its compute capability (**7.5** for a T4):

**Colab CODE cell · checks the GPU**

```
!nvidia-smi
!nvidia-smi --query-gpu=name,compute_cap --format=csv
```

4. Every program is one cell that starts with `%%writefile name.cu` (this must be the **first line** of the cell), followed by a second cell that compiles and runs it. All compile commands in this sheet use `-arch=sm_75` (T4). For a different GPU, remove the dot from the compute capability, for example 8.9 becomes `-arch=sm_89`.
5. The `.cu` files live on a temporary disk. If the runtime disconnects, reconnect and choose **Runtime → Run all**. When you finish, choose **Runtime → Disconnect and delete runtime** to release the GPU.

> **COMMON MISTAKE**
> **Edit → write → compile → run.** After changing any code you must re-run the `%%writefile` cell *and* the compile cell. Otherwise you are still running the old program.

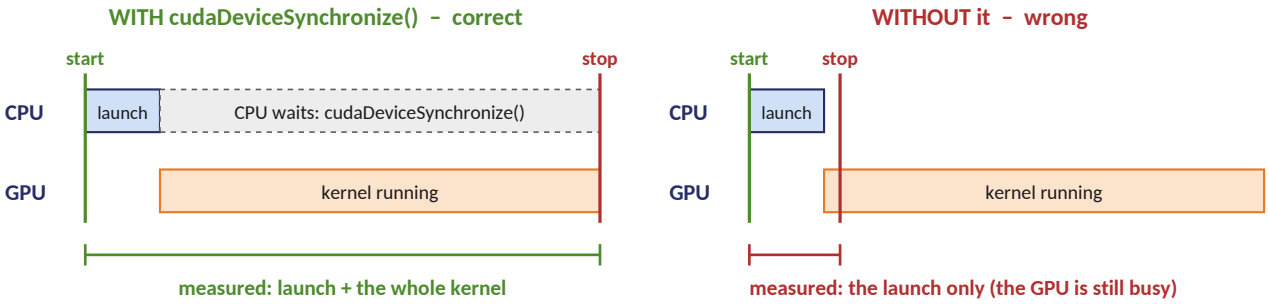
## Exercise 1 – Timing a GPU program

So far you have only checked that your kernels give the *right* answer. Now you will measure *how fast* they are. The program below is the 512-block vector addition from Lab 9 with a timer added. The timer uses `gettimeofday()`, which reads the CPU's wall clock in seconds.

- ☐ (a) Compile and run the timed program below.
- ☐ (b) Add the same timer to the programs you wrote in Lab 9 (Exercises 3, 4, 5 and 6) and record their times.
- ☐ (c) Compare the time for the kernel alone with the time for the whole copy-in, compute, copy-out sequence.

### Why the clock must wait for the GPU

A kernel launch is **asynchronous**: the CPU tells the GPU to start and carries on with the next line immediately. If you stop the clock straight after the launch, you have only timed the *launch*, not the work. `cudaDeviceSynchronize()` makes the CPU wait until the GPU has finished.




**Colab CODE cell · writes ex1_timed.cu**

```
%%writefile ex1_timed.cu
#include <cstdio>
#include <cstdlib>
#include <sys/time.h>

#define N 512

__global__ void add(int *a, int *b, int *c) {
    // each block runs only one addition
    c[blockIdx.x] = a[blockIdx.x] + b[blockIdx.x];
}

void random_ints(int *x, int size) {
    for (int i = 0; i < size; i++)
        x[i] = rand() % 100;
}

// Track CPU (wall-clock) time, in seconds
double cpuSecond() {
    struct timeval tp;
    gettimeofday(&tp, NULL);
    return (double)tp.tv_sec + (double)tp.tv_usec * 1.e-6;
}

int main(void) {
    int *a, *b, *c;             // host copies of a, b, c
    int *d_a, *d_b, *d_c;       // device copies of a, b, c
    int size = N * sizeof(int);
    double dStart, dElaps;

    // Allocate space for device copies of a, b, c
    cudaMalloc((void **)&d_a, size);
    cudaMalloc((void **)&d_b, size);
```

**Colab CODE cell · writes ex1_timed.cu**

```
    cudaMalloc((void **)&d_c, size);

    // Allocate space for host copies of a, b, c and set up input values
    a = (int *)malloc(size); random_ints(a, N);
    b = (int *)malloc(size); random_ints(b, N);
    c = (int *)malloc(size);

    // Copy inputs to device
    cudaMemcpy(d_a, a, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, size, cudaMemcpyHostToDevice);

    // initialize timer
    dStart = cpuSecond();

    // Launch add() kernel on GPU with N blocks
    add<<<N, 1>>>(d_a, d_b, d_c);
    cudaDeviceSynchronize();    // wait for the GPU to finish, THEN stop the clock

    dElaps = cpuSecond() - dStart;

    // Copy result back to host
    cudaMemcpy(c, d_c, size, cudaMemcpyDeviceToHost);

    for (int r = 0; r < 5; r++)
        printf("%d + %d = %d\n", a[r], b[r], c[r]);

    // Check the GPU's answers against the CPU
    int errors = 0;
    for (int r = 0; r < N; r++)
        if (c[r] != a[r] + b[r]) errors++;
    printf("Verification: %d mismatches out of %d\n", errors, N);

    printf("Time taken - %.7f\n", dElaps);

    // Cleanup
    free(a); free(b); free(c);
    cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
    return 0;
}
```

**Colab CODE cell · compiles and runs**

```
!nvcc -arch=sm_75 ex1_timed.cu -o ex1_timed && ./ex1_timed
```

The program prints only the first five results (the original sheet printed all 512) and then checks all 512 on the CPU. Expected output:

**expected output (the time changes from run to run)**

```
83 + 77 = 160
86 + 34 = 120
77 + 90 = 167
15 + 26 = 41
93 + 24 = 117
Verification: 0 mismatches out of 512
Time taken - 0.0000xxx
```

**EXPERIMENT – remove the synchronise**

In the `ex1_timed.cu` cell delete the line `cudaDeviceSynchronize();`, re-run the cell, compile and run again. What happens to **Time taken**? Does the answer still verify correctly? Explain both results using the picture above. Put the line back afterwards.

## Part (b) – time your Lab 9 programs

Copy each Lab 9 program into a new cell with a new file name (for example `ex3_blocks_timed.cu`) so you keep the original. Then add three things:

**add these pieces to each program**

```
#include <sys/time.h>                    // 1. at the top of the file

double cpuSecond() {                     // 2. above main()
    struct timeval tp;
    gettimeofday(&tp, NULL);
    return (double)tp.tv_sec + (double)tp.tv_usec * 1.e-6;
}

double dStart, dElaps;                   // 3. inside main(), around the launch
dStart = cpuSecond();
/* your existing kernel launch line */
cudaDeviceSynchronize();
dElaps = cpuSecond() - dStart;
printf("Time taken - %.7f\n", dElaps);
```

Run each program **three times** and write the times in the table. Timings vary a little from run to run, so use all three.

| Lab 9 program | Launch configuration | Run 1 (s) | Run 2 (s) | Run 3 (s) |
|---|---|---|---|---|
| **Exercise 3** | 512 blocks × 1 thread | | | |
| **Exercise 4** | 1 block × 512 threads | | | |
| **Exercise 5** | 10,000,000 elements, 19,532 blocks × 512 threads | | | |
| **Exercise 6** | 100,000,000 elements, 313 × 625 blocks of 32 × 16 | | | |

**PREDICT BEFORE YOU MEASURE**

1. Exercise 3 uses 512 blocks of one thread; Exercise 4 uses one block of 512 threads. Which do you expect to be faster, and why?

2. Exercise 6 has 10 times as many elements as Exercise 5. Do you expect it to take about 10 times as long? Look at your table, then explain any difference.

## Part (c) – kernel time versus copy time

In your timed Lab 9 Exercise 5 program change `#define N 10000000` to `#define N 100000000` (100 million elements, about 400 MB per array). Record two times:

1. the **kernel only** (the timer as you already have it), and
2. the **whole sequence**: start the clock before the first `cudaMemcpy(... HostToDevice)` and stop it after the last `cudaMemcpy(... DeviceToHost)`.

**THINK**

Which is larger, and by roughly how much? What does this tell you about the kind of work that is worth sending to a GPU? (Hint: the data has to travel over the PCIe bus in both directions.)

**EXERCISE 1 CHECKLIST**

- ☐ The timed program prints `Verification: 0 mismatches out of 512` and a time.
- ☐ You can explain what happens to the time when `cudaDeviceSynchronize()` is removed.
- ☐ The table has three times for each of the four Lab 9 programs.
- ☐ You have recorded kernel-only and whole-sequence times for 100 million elements.

## Exercise 2 – Catching errors with the CHECK() macro

Almost every CUDA runtime function returns a `cudaError_t` value. In Lab 9 you checked one launch by hand with `cudaGetLastError()`. The `CHECK()` macro does the checking for **every** call, and when something fails it prints the file, the line number, the error code and the reason, then stops the program.

| Where the error happens | How you catch it |
|---|---|
| Any CUDA call that returns a value: `cudaMalloc`, `cudaMemcpy`, `cudaFree` | Wrap the call: `CHECK(cudaMalloc(...));` |
| The kernel **launch** is wrong (for example too many threads per block) | A launch returns nothing, so add `CHECK(cudaGetLastError());` straight after it |
| The kernel **fails while running** (for example an illegal memory access) | Wrap the synchronise: `CHECK(cudaDeviceSynchronize());` |

Incorporate `CHECK()` into the program from Exercise 1. The starter below already contains the macro and marks every call that needs wrapping with `TODO`. It compiles and runs as it is, but does not yet check anything.

**Colab CODE cell · writes ex2_check.cu (edit the TODO lines)**

```
%%writefile ex2_check.cu
#include <cstdio>
#include <cstdlib>
#include <sys/time.h>

#define N 512

// Checks the value returned by a CUDA runtime call. On failure it prints the
// file, line number, error code and reason, then stops the program.
#define CHECK(call)                                                    \
    do {                                                               \
        const cudaError_t error = (call);                              \
        if (error != cudaSuccess) {                                    \
            printf("Error: %s:%d, ", __FILE__, __LINE__);              \
            printf("code:%d, reason: %s\n", error,                     \
                   cudaGetErrorString(error));                         \
            exit(1);                                                   \
        }                                                              \
    } while (0)

__global__ void add(int *a, int *b, int *c) {
    c[blockIdx.x] = a[blockIdx.x] + b[blockIdx.x];
}

void random_ints(int *x, int size) {
    for (int i = 0; i < size; i++)
        x[i] = rand() % 100;
}

double cpuSecond() {
    struct timeval tp;
    gettimeofday(&tp, NULL);
    return (double)tp.tv_sec + (double)tp.tv_usec * 1.e-6;
}
```

**Colab CODE cell · writes ex2_check.cu (edit the TODO lines)**

```
int main(void) {
    int *a, *b, *c;             // host copies of a, b, c
    int *d_a, *d_b, *d_c;       // device copies of a, b, c
    int size = N * sizeof(int);
    double dStart, dElaps;

    // TODO 1: wrap each of the three cudaMalloc calls in CHECK(...)
    cudaMalloc((void **)&d_a, size);
    cudaMalloc((void **)&d_b, size);
    cudaMalloc((void **)&d_c, size);

    a = (int *)malloc(size); random_ints(a, N);
    b = (int *)malloc(size); random_ints(b, N);
    c = (int *)malloc(size);

    // TODO 2: wrap both host-to-device cudaMemcpy calls in CHECK(...)
    cudaMemcpy(d_a, a, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, size, cudaMemcpyHostToDevice);

    dStart = cpuSecond();
    add<<<N, 1>>>(d_a, d_b, d_c);
    // TODO 3: add CHECK(cudaGetLastError()); here, and wrap cudaDeviceSynchronize()
    cudaDeviceSynchronize();
    dElaps = cpuSecond() - dStart;

    // TODO 4: wrap the device-to-host cudaMemcpy in CHECK(...)
    cudaMemcpy(c, d_c, size, cudaMemcpyDeviceToHost);

    for (int r = 0; r < 5; r++)
        printf("%d + %d = %d\n", a[r], b[r], c[r]);

    int errors = 0;
    for (int r = 0; r < N; r++)
        if (c[r] != a[r] + b[r]) errors++;
    printf("Verification: %d mismatches out of %d\n", errors, N);
    printf("Time taken - %.7f\n", dElaps);

    free(a); free(b); free(c);
    // TODO 5: wrap the three cudaFree calls in CHECK(...)
    cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
    return 0;
}
```

**Colab CODE cell · compiles and runs**

```
!nvcc -arch=sm_75 ex2_check.cu -o ex2_check && ./ex2_check
```

When all five TODOs are done, the output is identical to Exercise 1. A macro that prints nothing is easy to trust too much, so now **break the program on purpose** and check that `CHECK()` notices.

**BREAK IT ON PURPOSE**

**Experiment A – too many threads.** Change the launch to `add<<<N, 2048>>>(d_a, d_b, d_c);` (the limit is 1024 threads per block). Expected: `Error: ex2_check.cu:<line>, code:9, reason: invalid configuration argument` and the program stops.

**Experiment B – impossible allocation.** Change the first `cudaMalloc` size to `(size_t)1 << 50` (one petabyte). Expected: `code:2, reason: out of memory`.

**Experiment C – no CHECK.** Try Experiment A on the plain Exercise 1 program. What does it print? How would you know something had gone wrong if the verification loop were not there?

Change the program back after each experiment.

**WHY do { ... } while (0)?**

Wrapping the macro body in `do { ... } while (0)` makes `CHECK(x);` behave like one ordinary statement, even inside an `if` without braces. The sheet's earlier version used plain braces, which works in most places but not there.

**EXERCISE 2 CHECKLIST**

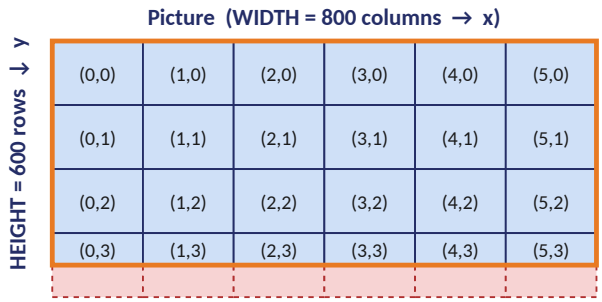
- ☐ All five TODOs are done and the output matches Exercise 1.
- ☐ Experiment A printed code 9 and Experiment B printed code 2.
- ☐ You can say which of the three rows of the table above each experiment tested.

## Exercise 3 – Scaling a picture with a 2D grid

Solve the *multidimensional grid* example from the lecture: the **PictureKernel** scales every pixel of a picture by 2.0. Your picture is 800 pixels wide and 600 pixels high.

- ☐ (a) Represent the picture as a **single-dimension array** of 800 × 600 = 480,000 elements of type `unsigned char`.
- ☐ (b) Give every element a random value between 0 and 255.
- ☐ (c) Use the PictureKernel from the slide (given in the starter).
- ☐ (d) Use `dim3` variables to define 16 threads per block in x and y, and a grid with enough blocks for the whole picture.
- ☐ (e) Launch the kernel using those two `dim3` variables.
- ☐ (f) Display a 50 × 50 portion of the picture with the original values and the updated values.

A picture in a one-dimensional array is stored **row by row**: pixel (Row, Col) sits at position `Row * width + Col`. Threads are arranged in 2D: the y coordinate selects the row and the x coordinate selects the column.




Not to scale. Each tile is one 16 × 16 block, labelled (blockIdx.x, blockIdx.y).

**Reading the picture**

Blue: threads on a real pixel

Red, dashed: threads below the picture (your guard must stop these)

**800 columns (x)**

800 / 16 = 50 → 50 blocks, exact fit

**600 rows (y)**

600 / 16 = 37.5 → 38 blocks

last block row: 8 rows inside, 8 outside

**WHY THE CAP AT 255?**

The slide's kernel works on `float` values. Your pixels are 8-bit, so the largest value is 255, and 2 × 200 = 400 does not fit. Storing 400 in an `unsigned char` would wrap around to 144 (400 − 256), which is a wrong, *darker* pixel. The kernel in the starter therefore **caps** the result at 255 (this is called *saturation*). Your CPU check in TODO 6 must apply the same rule.

Complete the starter program. Each place you must change is marked with `TODO` and highlighted. The kernel itself is complete.

**Colab CODE cell · writes ex3_picture.cu (edit the TODO lines)**

```
%%writefile ex3_picture.cu
#include <cstdio>
#include <cstdlib>
#include <sys/time.h>

#define WIDTH  800              // columns
#define HEIGHT 600              // rows

#define CHECK(call)                                                    \
    do {                                                               \
        const cudaError_t error = (call);                              \
        if (error != cudaSuccess) {                                    \
            printf("Error: %s:%d, ", __FILE__, __LINE__);              \
            printf("code:%d, reason: %s\n", error,                     \
                   cudaGetErrorString(error));                         \
            exit(1);                                                   \
        }                                                              \
    } while (0)

double cpuSecond() {
    struct timeval tp;
    gettimeofday(&tp, NULL);
    return (double)tp.tv_sec + (double)tp.tv_usec * 1.e-6;
}

// Kernel from the lecture slide "Source Code of a PictureKernel", adapted
// for 8-bit pixels: scale every pixel value by 2.0, but never exceed 255.
__global__ void PictureKernel(unsigned char *d_Pin, unsigned char *d_Pout,
                              int height, int width) {
    // Calculate the row # of the d_Pin and d_Pout element
    int Row = blockIdx.y * blockDim.y + threadIdx.y;
    // Calculate the column # of the d_Pin and d_Pout element
    int Col = blockIdx.x * blockDim.x + threadIdx.x;

    // each thread computes one element of d_Pout if in range
    if ((Row < height) && (Col < width)) {
        int v = 2 * d_Pin[Row * width + Col];
        d_Pout[Row * width + Col] = (v > 255) ? 255 : v;
    }
}

int main(void) {
    size_t count = (size_t)WIDTH * HEIGHT;      // ONE flat array: 480,000 pixels
    size_t size  = count * sizeof(unsigned char);
    unsigned char *h_in, *h_out;                // host: original and updated picture
    unsigned char *d_in, *d_out;                // device copies
    double dStart, dElaps;

    h_in  = (unsigned char *)malloc(size);
    h_out = (unsigned char *)malloc(size);
    CHECK(cudaMalloc((void **)&d_in,  size));
    CHECK(cudaMalloc((void **)&d_out, size));

    // TODO 1: give every pixel of h_in a random value between 0 and 255

    CHECK(cudaMemcpy(d_in, h_in, size, cudaMemcpyHostToDevice));

    // TODO 2: 16 threads per block in BOTH x and y
    dim3 threadsPerBlock(1, 1);                 // <-- replace this line
    // TODO 3: blocks needed in x (columns) and in y (rows), rounding UP
    dim3 numBlocks(1, 1);                       // <-- replace this line
    printf("Grid: %d x %d blocks of %d x %d threads\n",
           numBlocks.x, numBlocks.y, threadsPerBlock.x, threadsPerBlock.y);

    dStart = cpuSecond();
    // TODO 4: launch PictureKernel with <<<numBlocks, threadsPerBlock>>>
    CHECK(cudaGetLastError());
```

**Colab CODE cell · writes ex3_picture.cu (edit the TODO lines)**

```
    CHECK(cudaDeviceSynchronize());
    dElaps = cpuSecond() - dStart;

    CHECK(cudaMemcpy(h_out, d_out, size, cudaMemcpyDeviceToHost));

    // TODO 5: print the top-left 50 x 50 portion: first the original values,
    //         then the updated values (use printf("%3d ", ...) and a newline per row)

    // TODO 6: check every pixel on the CPU (expected = 2 x original, capped at 255),
    //         count the mismatches, and count how many pixels were capped

    printf("Time taken - %.7f\n", dElaps);

    free(h_in); free(h_out);
    CHECK(cudaFree(d_in)); CHECK(cudaFree(d_out));
    return 0;
}
```

**Colab CODE cell · compiles and runs**

```
!nvcc -arch=sm_75 ex3_picture.cu -o ex3_picture && ./ex3_picture
```

**PREDICT BEFORE YOU RUN**

Roughly how many of the 480,000 pixels will be capped at 255? (Which original values become 255 when doubled?)

## Self-check: your program is correct if it reports these numbers

| Item | Expected value |
|---|---|
| **Grid printed by the starter** | `Grid: 50 x 38 blocks of 16 x 16 threads` (1,900 blocks) |
| **Total threads launched** | 486,400 (that is 6,400 more than the 480,000 pixels) |
| **Active rows in the last block row** | 8 of 16 (the other 8 must be stopped by your guard) |
| **First ten original values (row 0)** | `103 198 105 115 81 255 74 236 41 205` |
| **First ten updated values (row 0)** | `206 255 210 230 162 255 148 255 82 255` |
| **Verification** | `0 mismatches out of 480000` |
| **Pixels capped at 255** | about half of them (240,121 in our run) |

The values assume the pixels are generated in order with `rand() % 256`. Your print format may differ; compare the numbers.

**COMMON MISTAKE**

**Swapping x and y.** The x coordinate is the *column* (0 to 799) and the y coordinate is the *row* (0 to 599). If you swap them, the threads run past the picture in the wrong direction and the output is wrong.

**Rounding down.** `600 / 16` in integer arithmetic gives 37 blocks and leaves the last 8 rows of the picture untouched.

**Guarding only one dimension.** Both `Row < height` and `Col < width` must be checked. Without the row check the 8 extra rows of threads write past the end of the array.

**Forgetting the cap.** Without it the pixels above 127 wrap around and `Verification` reports about 240,000 mismatches.

**EXERCISE 3 CHECKLIST**

- ☐ The array is one-dimensional and holds 800 × 600 unsigned chars, each between 0 and 255.
- ☐ The launch uses two `dim3` variables and the grid is 50 × 38 blocks.
- ☐ The 50 × 50 original and updated portions are displayed, and verification reports 0 mismatches.

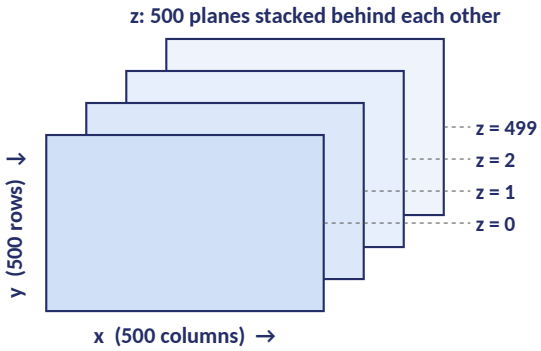
## Exercise 4 – Multiplying two 3D matrices element by element

Extend Lab 9 Exercise 6 from a 2D matrix to a **3D matrix** for `A`, `B` and `C`. As before, multiply corresponding elements, for example `C[4][3][2] = A[4][3][2] × B[4][3][2]`. This is not matrix multiplication.

- ☐ Use a 3D grid of blocks and 3D blocks of threads (`dim3` with three values).
- ☐ Print the **last 1000 elements** of C.
- ☐ Verify all elements on the CPU and print the time taken for the kernel.

**A NOTE ON THE SIZE**

The original sheet suggested 10,000 in every dimension. Do the sum: 10,000 × 10,000 × 10,000 = $10^{12}$ elements, which is **4 terabytes** for each matrix, far more than the 15 GB on a Colab GPU. This lab therefore uses **500 × 500 × 500** = 125,000,000 elements (500 MB per matrix; about 1.5 GB on the GPU and 1.5 GB in Colab's host memory for the three matrices).




**How the volume sits in memory**

Each plane is a 500 × 500 picture.
x changes fastest, then y, then z.

**Flat index of C[z][y][x]**

```
(z * 500 + y) * 500 + x
```

Last element: C[499][499][499]
has flat index 124,999,999.

The pieces are the same as in 2D, with one more coordinate:

| What | Code |
|---|---|
| **Column (x) of this thread** | `x = blockIdx.x * blockDim.x + threadIdx.x` |
| **Row (y) of this thread** | `y = blockIdx.y * blockDim.y + threadIdx.y` |
| **Plane (z) of this thread** | `z = blockIdx.z * blockDim.z + threadIdx.z` |
| **Flat index** | `idx = (z * ny + y) * nx + x` |
| **Threads per block** | `dim3 threadsPerBlock(8, 8, 8);` → 8 × 8 × 8 = 512 threads |
| **Blocks in the grid** | `dim3 numBlocks(bx, by, bz);` (round all three up) |
| **Launch** | `mulElem3D<<<numBlocks, threadsPerBlock>>>(...)` |

**COMMON MISTAKE**

**Too many threads per block.** A block of 16 × 16 × 16 has 4096 threads, but the limit is 1024. The launch fails, and `CHECK()` reports `code:9, reason: invalid configuration argument`. 8 × 8 × 8 = 512 is safe.

**Guarding only two of the three coordinates.** All of `x < nx`, `y < ny` and `z < nz` must be checked.

**A flat-index formula that is not one-to-one.** Every (x, y, z) needs its own position. A formula such as `(z * ny + y) + x` sends many threads to the same element and leaves most of C uncomputed; the verification then reports millions of mismatches.

**Declaring the matrix as a local array** such as `int A[500][500][500];`. It needs 500 MB and overflows the stack. Use `malloc` for one flat array, as the starter does.

**Running out of memory.** If Colab reports that the session crashed after using all available RAM, restart the runtime and check that you have not made extra copies of the arrays.

Complete the starter program. The timer and the `CHECK()` macro from Exercises 1 and 2 are already in place.

**Colab CODE cell · writes ex4_volume.cu (edit the TODO lines)**

```
%%writefile ex4_volume.cu
#include <cstdio>
#include <cstdlib>
#include <sys/time.h>

#define NX 500                  // x: columns
#define NY 500                  // y: rows
#define NZ 500                  // z: planes (depth)

#define CHECK(call)                                                    \
    do {                                                               \
        const cudaError_t error = (call);                              \
        if (error != cudaSuccess) {                                    \
            printf("Error: %s:%d, ", __FILE__, __LINE__);              \
            printf("code:%d, reason: %s\n", error,                     \
                   cudaGetErrorString(error));                         \
            exit(1);                                                   \
        }                                                              \
    } while (0)

double cpuSecond() {
    struct timeval tp;
    gettimeofday(&tp, NULL);
    return (double)tp.tv_sec + (double)tp.tv_usec * 1.e-6;
}

// Element-wise product: C[z][y][x] = A[z][y][x] * B[z][y][x]
__global__ void mulElem3D(int *a, int *b, int *c, int nx, int ny, int nz) {
    // TODO 1: the x, y and z coordinates of this thread
    int x = 0;                                  // <-- replace this line
    int y = 0;                                  // <-- replace this line
    int z = 0;                                  // <-- replace this line

    // TODO 2: only compute if x, y AND z are all inside the volume
    // TODO 3: flat index of (x, y, z):  (z * ny + y) * nx + x
    size_t idx = 0;                             // <-- replace this line
    c[idx] = a[idx] * b[idx];                   // <-- guard this line
}

void random_ints(int *x, size_t size) {
    for (size_t i = 0; i < size; i++)
        x[i] = rand() % 100;
}

int main(void) {
    size_t count = (size_t)NX * NY * NZ;        // 125,000,000 elements
    size_t size  = count * sizeof(int);         // 500 MB per array
    int *a, *b, *c;                             // host copies (flat arrays)
```

**Colab CODE cell · writes ex4_volume.cu (edit the TODO lines)**

```
    int *d_a, *d_b, *d_c;                       // device copies
    double dStart, dElaps;

    CHECK(cudaMalloc((void **)&d_a, size));
    CHECK(cudaMalloc((void **)&d_b, size));
    CHECK(cudaMalloc((void **)&d_c, size));

    a = (int *)malloc(size); random_ints(a, count);
    b = (int *)malloc(size); random_ints(b, count);
    c = (int *)malloc(size);

    CHECK(cudaMemcpy(d_a, a, size, cudaMemcpyHostToDevice));
    CHECK(cudaMemcpy(d_b, b, size, cudaMemcpyHostToDevice));

    // 8 x 8 x 8 = 512 threads per block
    dim3 threadsPerBlock(8, 8, 8);
    // TODO 4: blocks needed in x, y and z, rounding UP
    dim3 numBlocks(1, 1, 1);                    // <-- replace this line
    printf("Grid: %d x %d x %d blocks\n", numBlocks.x, numBlocks.y, numBlocks.z);

    dStart = cpuSecond();
    // TODO 5: launch mulElem3D with <<<numBlocks, threadsPerBlock>>>
    CHECK(cudaGetLastError());
    CHECK(cudaDeviceSynchronize());
    dElaps = cpuSecond() - dStart;

    CHECK(cudaMemcpy(c, d_c, size, cudaMemcpyDeviceToHost));

    // TODO 6: print the last 1000 elements of C, in memory order
    //         (C[499][498][0] ... C[499][499][499]) as "C[z][y][x] = a x b = c"

    // TODO 7: check all elements on the CPU and print the number of mismatches

    printf("Time taken - %.7f\n", dElaps);

    free(a); free(b); free(c);
    CHECK(cudaFree(d_a)); CHECK(cudaFree(d_b)); CHECK(cudaFree(d_c));
    return 0;
}
```

**Colab CODE cell · compiles and runs**

```
!nvcc -arch=sm_75 ex4_volume.cu -o ex4_volume && ./ex4_volume
```

## Self-check: your program is correct if it reports these numbers

| Item | Expected value |
|---|---|
| **Grid printed by the starter** | `Grid: 63 x 63 x 63 blocks` (250,047 blocks) |
| **Total threads launched** | 128,024,064 (that is 3,024,064 more than the 125,000,000 elements) |
| **First of the last 1000 results** | `C[499][498][0] = 76 x 15 = 1140` |
| **Last result** | `C[499][499][499] = 47 x 68 = 3196` |
| **Verification** | `0 mismatches out of 125000000` |

The numbers assume `a` is generated first and `b` second with the starter's `random_ints` function. The program takes several seconds to fill 250 million random numbers before the GPU is even used.

**THINK**

Lab 9 Exercise 6 (2D, 100 million elements) and this exercise (3D, 125 million elements) do the same work per element. Compare your **Time taken** values. Is the 3D kernel faster, slower or about the same? Why might the arrangement of threads matter so little, or so much, here?

**FINISHING OFF**

- ☐ Your notebook has a text heading and the working cells for every exercise, with the outputs visible.
- ☐ Use **File → Save** (Colab also saves to Drive automatically).
- ☐ Submit or share the notebook as your lecturer instructs (**File → Download → Download .ipynb** or **Share**).
- ☐ Choose **Runtime → Disconnect and delete runtime** to release the GPU.

## Quick reference

| Item | What it does |
|---|---|
| `gettimeofday(&tp, NULL)` | Reads the CPU wall clock into a `struct timeval` (needs `#include <sys/time.h>`). |
| `cpuSecond()` | Helper from this lab: returns the current time in seconds as a `double`. Subtract two readings to get a duration. |
| `cudaDeviceSynchronize()` | Makes the CPU wait until all GPU work has finished. Needed before you stop a timer. |
| `cudaGetLastError()` | Returns the error from the most recent kernel launch, or `cudaSuccess`. |
| `CHECK(call)` | Runs `call`; if it fails, prints file, line, code and reason, then exits. |
| `cudaGetErrorString(err)` | Turns an error code into readable text. |
| `dim3 v(x, y, z)` | A triple of unsigned integers used for grid and block sizes. Missing values default to 1. |
| `kernel<<<grid, block>>>(args)` | Launches with `grid` blocks of `block` threads. Both can be `dim3`. |
| `threadIdx.{x,y,z}` | Position of the thread *inside* its block. |
| `blockIdx.{x,y,z}` | Position of the block inside the grid. |
| `blockDim.{x,y,z}` | Number of threads per block in each dimension (at most 1024 threads in total). |
| `unsigned char` | 8-bit value from 0 to 255; used for the picture pixels. |

## Troubleshooting in Colab

| What you see | Likely cause and fix |
|---|---|
| `nvidia-smi` fails, or `no kernel image is available` | No GPU in the runtime (**Runtime → Change runtime type**) or the `-arch` flag does not match your GPU. |
| Your edit has no effect | You changed the code but did not re-run the `%%writefile` cell and then the compile cell. |
| Time is tiny and does not change when the problem gets bigger | The clock stopped before the GPU finished: `cudaDeviceSynchronize()` is missing or is after `dElaps` is calculated. |
| `Time taken - 0.0000000` | The kernel was never launched (an unfinished TODO), or the time is below the timer's resolution. Check the launch line. |
| The time changes a lot between runs | Normal for very short kernels. Run three times and compare all three, as in the Exercise 1 table. |
| `CHECK` prints `code:9, reason: invalid configuration argument` | More than 1024 threads per block, or a grid dimension of 0. In Exercise 4, use 8 × 8 × 8, not 16 × 16 × 16. |
| `CHECK` prints `code:2, reason: out of memory` | Too much GPU memory requested, or an old program is still holding memory. Use **Runtime → Restart session**. |
| Picture output is wrong or the bottom rows are unchanged | x and y are swapped, or the number of blocks in y was rounded down (37 instead of 38). |
| Verification reports about 240,000 mismatches (Exercise 3) | The result was not capped at 255 on the CPU or on the GPU; both must use the same rule. |
| “Your session crashed after using all available RAM” (Exercise 4) | The matrices were declared as local arrays or copied unnecessarily. Use flat `malloc` arrays and restart the runtime. |
| Exercise 4 seems to hang for several seconds before printing | Normal: the CPU is generating 250 million random numbers. Wait for the output. |
| Runtime disconnected and files are gone | Colab's disk is temporary. Reconnect and choose **Runtime → Run all**. |